12

KARPLUS-STRONG ON A RASPBERRY PI PICO

In **Chapter 4**, you learned how to make plucked string sounds using the Karplus-Strong algorithm. You saved the generated sounds as WAV files and played notes from a pentatonic musical scale on your computer. In this chapter, you'll learn how to shrink that project to fit on a tiny piece of hardware: the Raspberry Pi Pico.

The Pico (see **Figure 12-1**) is built using an RP2040 microcontroller chip, which has just 264KB of random access memory (RAM). Compare that to the tens of gigabytes of RAM on the typical personal computer! The Pico also has 2MB of flash memory on a separate chip, in contrast to a normal computer's hundreds of gigabytes of hard disk space. Despite these limitations, however, the Pico is still extremely capable. It can perform many useful services, while also being much cheaper and less power-hungry than a regular computer. Your watch, your air conditioning unit, your clothes dryer, your car, your phone—tiny microcontrollers like the RP2040 are everywhere!

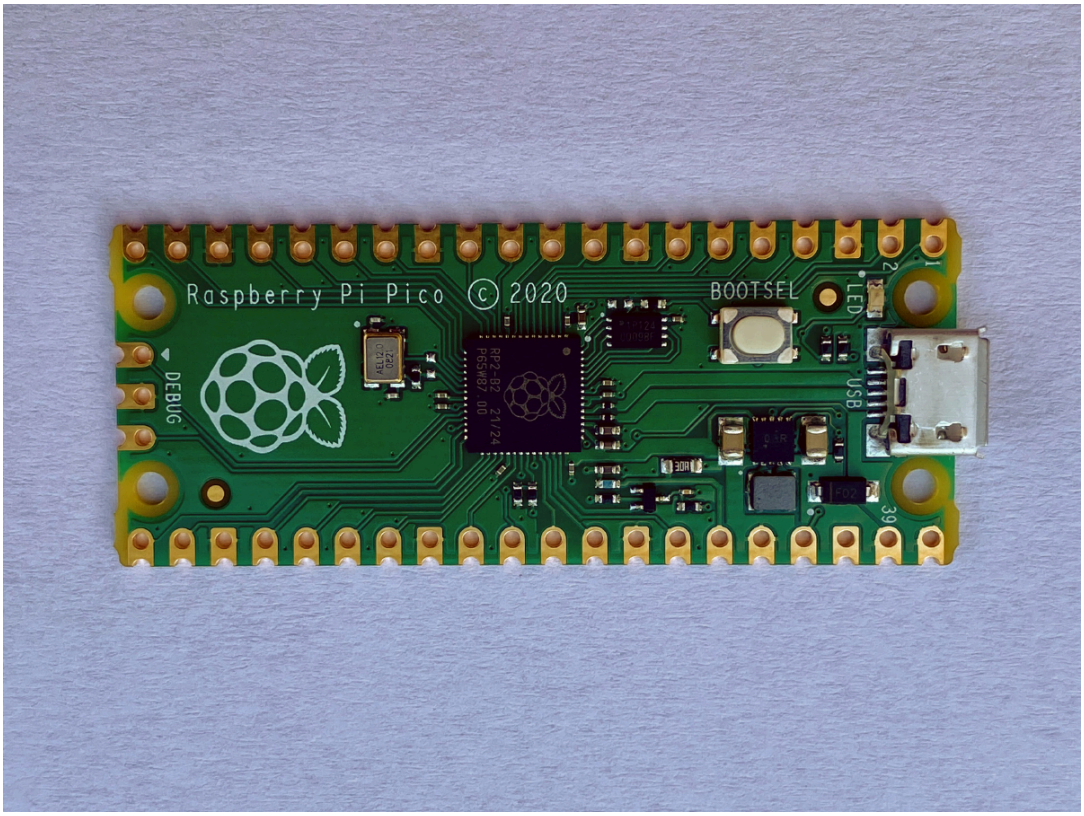

Figure 12-1: *The Raspberry Pi Pico*

The goal for this project is to use the Raspberry Pi Pico to create a musical instrument with five buttons. Pressing each button will play a note from a pentatonic scale, generated with the Karplus-Strong algorithm. Some of the concepts you'll learn from this project are:

- Programming a microcontroller using MicroPython, an implementation of Python optimized to run on devices like the Pico
- Building a simple audio circuit on a breadboard using the Pico
- Using the I2S digital audio protocol and an I2S amplifier to send audio data to a speaker
- Implementing the Karplus-Strong algorithm from **Chapter 4** on a resource-constrained microcontroller

How It Works

We discussed the Karplus-Strong algorithm in detail in **Chapter 4**, so we won't revisit it here. Instead, we'll focus on what makes this ver-

sion of the project different. Your program from **Chapter 4** was designed to run on a laptop or desktop computer. Thanks to the computer's ample RAM and hard disk resources, it had no problems creating WAV files using the Karplus-Strong algorithm and playing audio through speakers with `pyaudio`. The challenge now is to fit the project code onto a resource-constrained Raspberry Pi Pico. This will require the following modifications:

- Using a smaller audio sampling rate to reduce memory requirements
- Using a simple binary file to store raw generated samples rather than a WAV file
- Using the I2S protocol to send out the audio data to an external audio amplifier
- Using memory management techniques to avoid copying the same data repeatedly

We'll discuss the specifics of these modifications as they arise.

Input and Output

To make the project interactive, you'll want the Pico to generate sounds in response to user input. You'll need to wire five push buttons to the Pico for this purpose, since the Pico doesn't have a keyboard or mouse. (You'll use a sixth push button to run the program.) We also need to figure out how to produce the sound output, since unlike a personal computer, the Pico board doesn't have any built-in speakers. **Figure 12-2** shows a block diagram of the project.

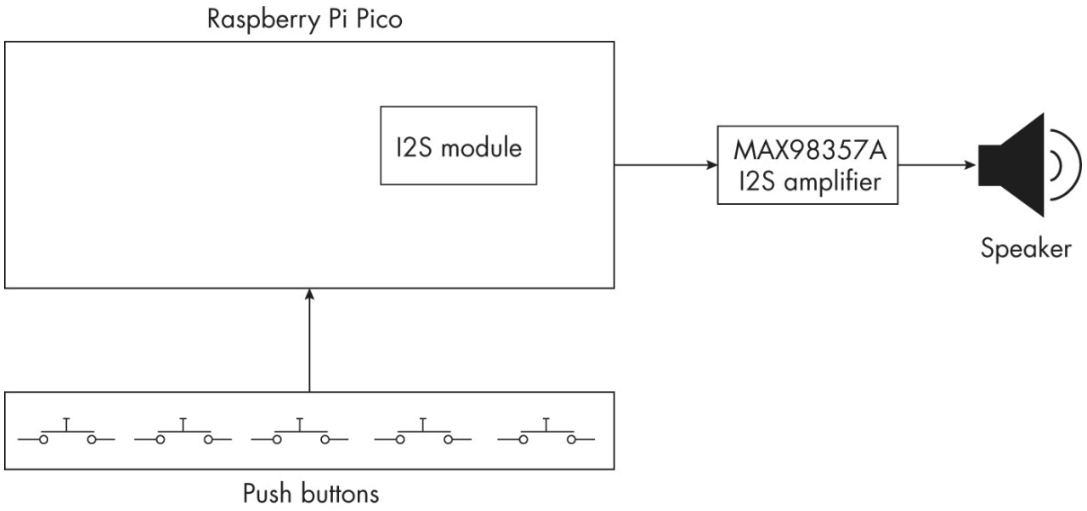

Figure 12-2: *A block diagram of the project*

When you press a button, the MicroPython code running on the Pico will generate a plucked string sound using the Karplus-Strong algorithm. The digital sound samples produced by the algorithm will be sent to a separate MAX98357A amplifier board, which decodes the digital data into an analog audio signal. The MAX98357A also amplifies the analog signal, which allows you to connect its output to an external 8-ohm speaker so you can hear the audio. **Figure 12-3** shows the Adafruit MAX98357A board.

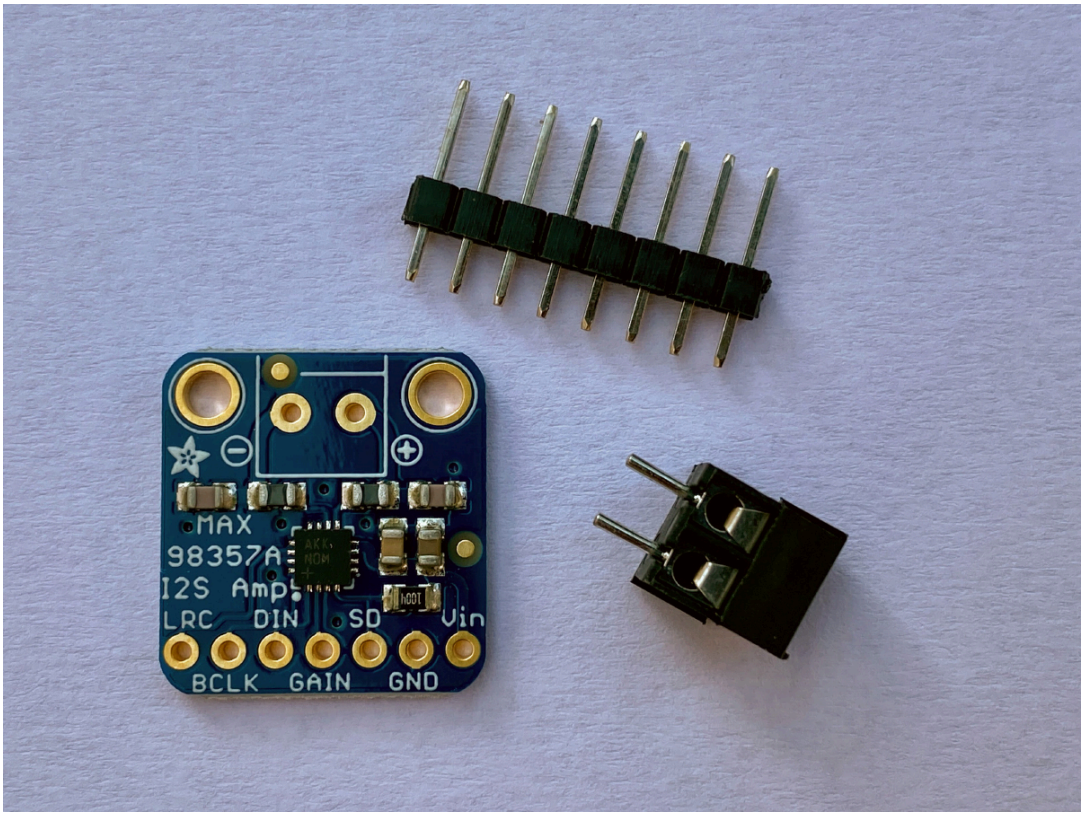

Figure 12-3: *An Adafruit MAX98357A I2S amplifier board*

The Pi Pico needs to send data to the amplifier board in a certain format for it to successfully be interpreted as an audio signal. Enter the I2S protocol.

The I2S Protocol

The *Inter-IC Sound (I2S) protocol* is a standard for sending digital audio data between devices. It's a simple, convenient way to get quality audio output from a microcontroller. The protocol transmits audio using three digital signals, which are shown in **Figure 12-4**.

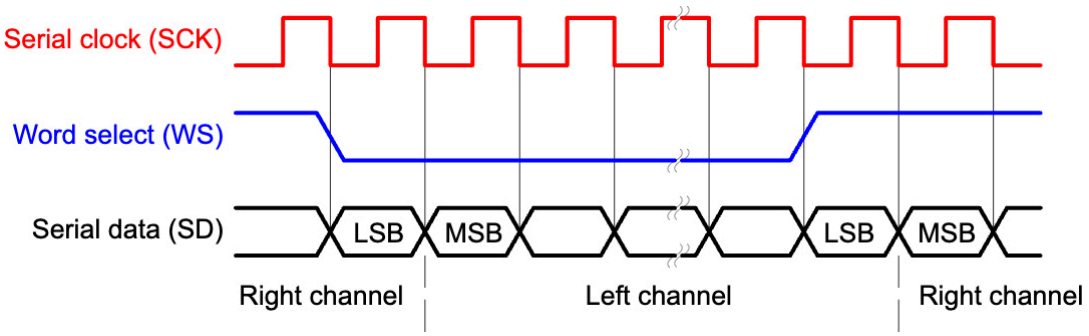

Figure 12-4: *The I2S protocol*

The first signal, SCK, is the *clock*, a signal that alternates between high and low at a fixed speed. This sets the rate of data transmission. Next, WS is the *word select* signal. It steadily alternates between high and low to indicate which audio channel, left or right, is being sent at any given moment. Finally, SD is the *serial data* signal, which carries the actual audio information, in the form of N-bit binary values representing the amplitude of the sound.

To understand how this works, let's consider an example. Say you want to send stereo audio at a sampling rate of 16,000 Hz and you want the amplitude of each sound sample to be a 16-bit value. The frequency of WS should be the same as the sampling rate, since that's the rate at which you're sending each amplitude value. This way, the WS signal will alternate between high and low 16,000 times per second; when it's high, SD will send the amplitude value of one audio channel, and when it's low, SD will send the amplitude value of the other audio channel. Since each amplitude value for each channel is made up of 16 bits, SD has to transmit at a rate that's 16 × 2 = 32 times faster than the sampling rate. The clock controls the rate of transmission, so SCK's frequency must be 16,000 Hz × 32 = 512,000 Hz.

For this project, the Pico will be the I2S transmitter, so it will generate the SCK, WS, and SD signals. MicroPython actually has a fully implemented `I2S` module for the Pico, so much of the work generating the signals will be done for you, behind the scenes. As you've already seen, the Pico will send the signals to a MAX98357A board, which is

specifically designed to receive audio data through the I2S protocol. Then the board converts the I2S data into an analog audio signal that can be played through a speaker.

Requirements

You'll program the project for the Raspberry Pi Pico using MicroPython. You'll need the following hardware:

- One Raspberry Pi Pico board based on the RP2040 chip
- One Adafruit MAX98357A I2S breakout board
- One 8-ohm speaker
- Six push buttons
- Five 10 kΩ resistors
- One breadboard
- An assortment of hookup wires
- One Micro USB cable for uploading code to the Pico

Hardware Setup

You'll assemble the hardware on a breadboard. **Figure 12-5** shows the hookup.

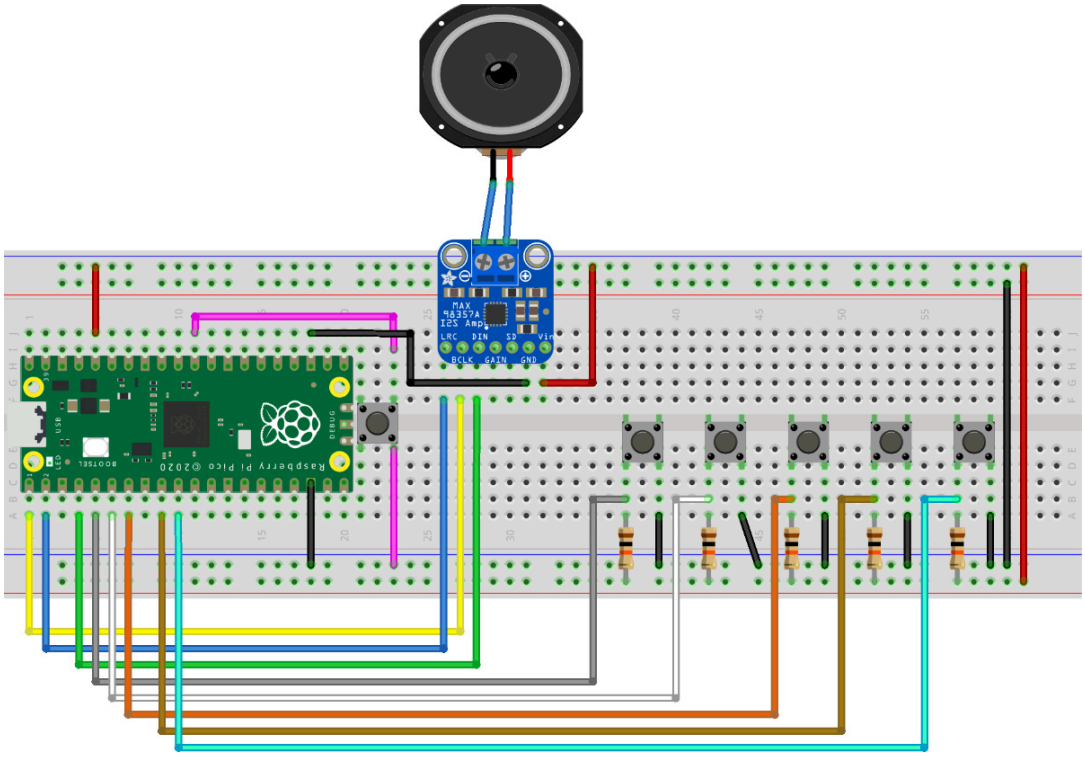

Figure 12-5: The hardware hookup

Figure 12-6 shows a pin diagram of the Pico from the official datasheet, which is a handy reference for your hookup.

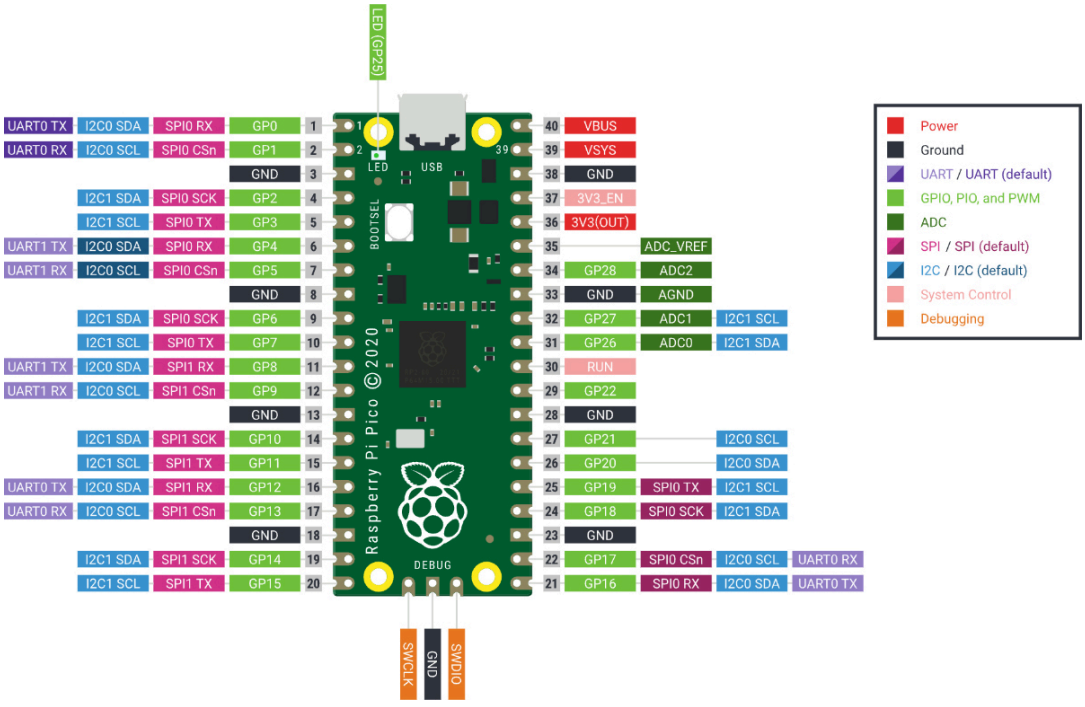

Figure 12-6: A pin diagram from the Raspberry Pi Pico datasheet

Table 12-1 summarizes the electrical connections you need to implement on the breadboard. **Figure 12-5** shows these connections.

Table 12-1: Electrical Connections

Pico pin	Connection
GP3	Push button 1 (other pin to VDD via 10 kΩ resistor)
GP4	Push button 2 (other pin to VDD via 10 kΩ resistor)
GP5	Push button 3 (other pin to VDD via 10 kΩ resistor)
GP6	Push button 4 (other pin to VDD via 10 kΩ resistor)
GP7	Push button 5 (other pin to VDD via 10 kΩ resistor)
RUN	Push button 6 (other pin to GND)
GP0	MAX98357A BCLK
GP1	MAX98357A LRC
GP2	MAX98357A DIN
GND	MAX98357A GND
3V3(OUT)	MAX98357A Vin

Once you've hooked up the hardware, your project should look like **Figure 12-7**.

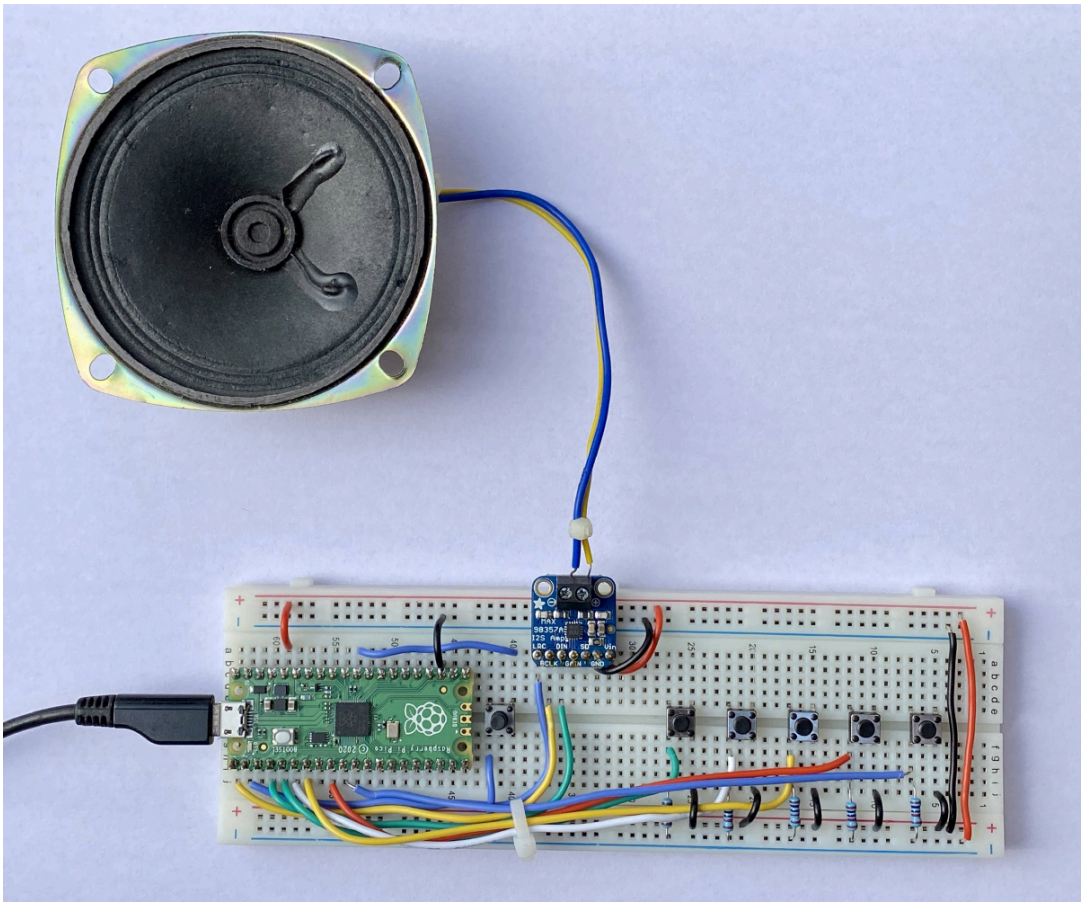

Figure 12-7: *The fully built hardware*

Before you start using your Pico, however, you need to set up MicroPython.

MicroPython Setup

Setting up your Raspberry Pi Pico with MicroPython is quite straightforward. Follow these steps:

1. Visit ***https://micropython.org***, go to the Download page, and find the Raspberry Pi Pico.
2. Download the UF2 binary file (version 1.18 or later) containing the MicroPython implementation for the Pico.
3. Press the white BOOTSEL button on the Pico, and while holding this button down, use your Micro USB cable to connect the Pico to your computer. Then release the button.

4. You should see a folder called *RPI-RP2* pop up on your computer. Drag and drop the UF2 file into this folder.

Once the copying is done and the Pico reboots, you're all set to code the Pico using MicroPython!

The Code

The code consists of some initial setup, followed by functions for generating and playing the five notes. Then everything comes together in the program's `main()` function. To see the full program, skip ahead to **"The Complete Code"** on **page 275**. The code is also available on GitHub at ***https://github.com/mkvenkit/pp2e/blob/main/karplus_pico/karplus_pico.py***.

Setting Up

The code begins with some basic setup. First, import the required MicroPython modules:

```
import time

import array

import random

import os

from machine import I2S

from machine import Pin
```

You import the `time` module for its "sleep" functionality to create timed pauses during the execution of the code. The `array` module will let you create arrays for sending the sound data via I2S. An array is a more efficient version of a Python list, since it requires all members to

be of the same data type. You'll use the `random` module to fill the initial buffer with random values (the first step in the Karplus-Strong algorithm), and you'll use the `os` module to check if a note has already been saved in the filesystem. Finally, the `I2S` module will let you send sound data, and the `Pin` module lets you set up the pin outputs of the Pico.

You complete the setup by declaring some useful information:

```
# notes of a minor pentatonic scale

# piano C4-E(b)-F-G-B(b)-C5

❶ pmNotes = {'C4': 262, 'Eb': 311, 'F': 349, 'G':391, 'Bb':466}

# button to note mapping

❷ btnNotes = {0: ('C4', 262), 1: ('Eb', 311), 2: ('F', 349), 3: ('G', 391),

        4: ('Bb', 466)}

# sample rate

❸ SR = 16000
```

Here you define a dictionary `pmNotes` that maps the name of a note to its integer frequency value ❶. You'll use the names of the notes to save files containing the sound data, and you'll use the frequency values to generate the sounds using the Karplus-Strong algorithm. You also define a dictionary `btnNotes` that maps the ID of each push button (represented as the integers 0 through 4) to a tuple that has the corresponding note name and frequency value ❷. This dictionary controls which note is played when the user presses each button.

Finally, you define the sampling rate as 16,000 Hz ❸. This is the number of sound amplitude values per second that you'll be sending out

via I2S. Notice that this is much lower than the sampling rate of 44,100 Hz used in **Chapter 4**. This is because of the limited memory on the Pico compared to a standard computer.

Generating the Notes

You generate the five notes of the pentatonic scale with the help of two functions: `generate_note()` and `create_notes()`. The `generate_note()` function uses the Karplus-Strong algorithm to calculate the amplitude values for a single note, while `create_notes()` coordinates generating all five notes and saving their sample data to the Pico's filesystem. Let's consider the `generate_note()` function first. (You implemented a similar function in **Chapter 4**, so this might be a good time to review the original implementation.)

```
# generate note of given frequency

def generate_note(freq):

    nSamples = SR

    N = int(SR/freq)

    # initialize ring buffer

❶ buf = [2*random.random() - 1 for i in range(N)]

    # init sample buffer

❷ samples = array.array('h', [0]*nSamples)

    for i in range(nSamples):

      ❸ samples[i] = int(buf[0] * (2 ** 15 - 1))

      ❹ avg = 0.4975*(buf[0] + buf[1])
```

```
        buf.append(avg)

        buf.pop(0)

    ❺ return samples
```

The function starts by setting `nSamples`, the length of the samples buffer that will hold the final audio data, to `SR`, the sampling rate. Since `SR` is the number of samples per second, this implies that you'll be creating a one-second audio clip. Then you compute `N`, the number of samples in the Karplus-Strong ring buffer, by dividing the sampling rate by the frequency of the note being generated.

Next, you initialize your buffers. First you create the ring buffer with random initial values ❶. The `random.random()` method returns values in the range [0.0, 1.0], so `2*random.random() - 1` scales the values to the range [−1.0, 1.0]. Remember, you need both positive and negative amplitudes for the algorithm. Notice that you're implementing the ring buffer as a regular Python list, instead of as a `deque` object like you used in **Chapter 4**. MicroPython's `deque` implementation has restrictions and doesn't give you what you need for a ring buffer. Instead, you'll just use the regular `append()` and `pop()` list methods to add and remove elements from the buffer. You also create the samples buffer as an `array` object of length `nSamples` filled with zeros ❷. The `'h'` argument specifies that each element in this array is a *signed short*, a 16-bit value that can be positive or negative. Since each sample will be represented by a 16-bit value, this is exactly what you need.

Next, you iterate over the items in the `samples` array and build up the audio clip using the Karplus-Strong algorithm. You take the first sample value in the ring buffer and scale it from range [−1.0, 1.0] to range [−32767, 32767] ❸. (The range of a 16-bit signed short is [−32767, 32768]. You scale the amplitude values to be as high as possible, which will give you the highest possible volume of sound output.) Then you calculate an attenuated average of the first two samples in the ring

buffer ❹. (Here, `0.4975` is the same as `0.995*0.5` from the original implementation.) You use `append()` to add the new amplitude value to the end of the ring buffer, while using `pop()` to remove the first element, thus maintaining the buffer's fixed size. At the end of the loop, the samples buffer is full, so you return it for further processing ❺.

NOTE *Using `append()` and `pop()` to update the ring buffer works, but it isn't an efficient method of computation. We'll look more at optimization in* **"Experiments!"** *on* **page 273**.

Now let's consider the `create_notes()` function:

```
def create_notes():
    "create pentatonic notes and save to files in flash"
❶ files = os.listdir()
❷ for (k, v) in pmNotes.items():
    # set note filename
❸ file_name = k + ".bin"
    # check if file already exists
❹ if file_name in files:
        print("Found " + file_name + ". Skipping...")
        continue
    # generate note
    print("Generating note " + k + "...")
❺ samples = generate_note(v)
```

```
# write to file

print("Writing " + file_name + "...")

❻ file_samples = open(file_name, "wb")

❼ file_samples.write(samples)

❽ file_samples.close()
```

You don't want to have to run the Karplus-Strong algorithm every time the user presses a button to play a note, as that would be too slow. Instead, this function creates the notes the first time the code is run and stores them in the Pico's filesystem as *.bin* files. Then, as soon as the user presses a button, you'll be able to read the appropriate file and output the sound data via I2S.

You begin by using the `os` module to list the files on the Pico ❶. (There's no "hard disk" on the Pico. Rather, a flash chip on the Pico board is used to store data, and MicroPython provides a way to access this data like a normal filesystem.) Then you iterate through the items in the `pmNotes` dictionary, which maps note names to frequencies ❷. For each note, you generate a filename based on its name in the dictionary (for example, *C4.bin*) ❸. If a file with that name exists in the directory ❹, you've generated that note already, so you can skip to the next one. Otherwise, you generate the sound samples for that note using the `generate_note()` function ❺. Then you create a binary file with the appropriate name ❻ and write the samples to it ❼. Finally, you clean up by closing the file ❽.

The first time you run the code, `create_notes()` will run the `generate_note()` function to create a file for each note using the Karplus-Strong algorithm. This will create the files *C4.bin*, *Eb.bin*, *F.bin*, *G.bin*, and *Bb.bin* on the Pico. On subsequent runs, the function will find these files still in place, so it won't need to create them again.

Playing a Note

The `play_note()` function plays one of the notes from the pentatonic scale by outputting the samples using the I2S protocol. Here's the definition:

```
def play_note(note, audio_out):
    "read note from file and send via I2S"
❶ fname = note[0] + ".bin"
    print("opening " + fname)
    # open file
    try:
        print("opening {}...".format(fname))
    ❷ file_samples = open(fname, "rb")
    except:
        print("Error opening file: {}!".format(fname))
        return
    # allocate sample array
❸ samples = bytearray(1000)
    # memoryview used to reduce heap allocation
❹ samples_mv = memoryview(samples)
    # read samples and send to I2S
```

```
    try:
        ❺ while True:
            ❻ num_read = file_samples.readinto(samples_mv)
            # end of file?
            ❼ if num_read == 0:
                break
            else:
                # send samples via I2S
                ❽ num_written = audio_out.write(samples_mv[:num_read])
    ❾ except (Exception) as e:
        print("Exception: {}".format(e))
    # close file
    ❿ file_samples.close()
```

The function has two arguments: `note`, a tuple in the form `('C4', 262)` conveying the note name and frequency, and `audio_out`, an instance of the `I2S` module used for sound output. You first create the appropriate *.bin* filename based on the name of the note to be played ❶. Then you open the file ❷. You expect the file to exist at this point, so if the open fails, you just return from the function.

The rest of the function outputs the audio data via I2S, working in batches of 1,000 samples. To mediate the data transfer, you create a MicroPython `bytearray` of 1,000 samples ❸ and a `memoryview` of the samples ❹. This is a MicroPython optimization technique to prevent

the whole array from being copied when a slice of the array is passed into other functions such as `file_samples.readinto()` and `audio_out.write()`.

NOTE *A slice of an array represents a range of values within that array. For example, `a[100:200]` is a slice representing array values `a[100]` through `a[199]`.*

Next, you start a `while` loop to read samples from the file ❺. In the loop, you read a batch of samples from the file into the `memoryview` object using the `readinto()` method ❻, which returns the number of samples read (`num_read`). You output the samples from the `memoryview` object via I2S using the `audio_out.write()` method ❽. The `[:num_read]` slice notation ensures you write out the same number of samples you read in. You handle any exceptions at ❾. You're done outputting data when you get to the point where zero samples are read into the `memoryview` object ❼, in which case you can break out of the `while` loop and close the *.bin* file ❿.

Writing the main() Function

Now let's look at the `main()` function, which brings all the code together:

```
def main():
    # set up LED
❶ led = Pin(25, Pin.OUT)
    # turn on LED
    led.toggle()
    # create notes and save in flash
```

```
❷ create_notes()

 # create I2S object

❸ audio_out = I2S(

    0,              # I2S ID

    sck=Pin(0),       # SCK Pin

    ws=Pin(1),        # WS Pin

    sd=Pin(2),        # SD Pin

    mode=I2S.TX,      # I2S transmitter

    bits=16,          # 16 bits per sample

    format=I2S.MONO,   # Mono - single channel

    rate=SR,          # sample rate

    ibuf=2000,        # I2S buffer length

 )

 # set up btns

❹ btns = [Pin(3, Pin.IN, Pin.PULL_UP),

       Pin(4, Pin.IN, Pin.PULL_UP),

       Pin(5, Pin.IN, Pin.PULL_UP),

       Pin(6, Pin.IN, Pin.PULL_UP),

       Pin(7, Pin.IN, Pin.PULL_UP)]
```

```
    # "ready" note
❺ play_note(('C4', 262), audio_out)
    print("Piano ready!")
    # turn off LED
❻ led.toggle()
    while True:
        for i in range(5):
            if btns[i].value() == 0:
❼ play_note(btnNotes[i], audio_out)
                break
❽ time.sleep(0.2)
```

The function starts by setting up the Pico's onboard LED ❶. It's toggled ON at the start to indicate the Pico is busy initializing. Next, you call the `create_notes()` function ❷. As we discussed, this function will create the *.bin* files for the notes only if they don't already exist in the filesystem. To manage the audio output, you instantiate the `I2S` module as `audio_out` ❸. The module requires a number of input parameters. The first parameter is the I2S ID, which is `0` for the Raspberry Pi Pico. Next come the pin numbers corresponding to the clock (SCK), word select (WS), and data (SD) signals. We discussed these signals in **"I2S Protocol" on page 262**. You then set the I2S mode to `TX`, indicating this is an I2S transmitter. Next, you set `bits` to `16`, indicating the number of bits per sample, and `format` to `MONO`, since there's only one audio output channel. You set the sampling rate to `SR`, and lastly, you set the value for the internal I2S buffer `ibuf` to `2000`.

NOTE *A smooth audio experience requires an uninterrupted stream of data output. MicroPython uses a special hardware module in the Pico called Direct Memory Access (DMA) for this. DMA can transfer data from the memory to the I2S output without involving the CPU directly. The CPU just needs to keep an internal buffer (`ibuf` in the code) filled with data and is free to do other things while the DMA is doing its job. The size of the internal buffer is typically set to at least twice the size of the audio output so the DMA doesn't run out of data to transfer, which would result in distorted audio. In this case, you'll be transferring 1,000 bytes to I2S at a time, so you set `ibuf` to twice that.*

Next, you need to set up the buttons so that notes will play when the buttons are pressed. For this, you create a list of `Pin` objects called `btns` ❹. For each button in the list, you specify the pin number, the data direction of the pin (`Pin.IN`, or input, in this case), and whether the pin has a pull-up resistor. In this case, all the push button pins have a 10 kΩ resistor pull-up on them. This means that by default the pins' voltages are "pulled up" to VDD, or 3.3 V, and when the buttons are pushed, the voltage drops to GND, or 0 V. You'll use this fact to detect button presses.

Once the setup is done, you play a C4 note using the `play_note()` function to indicate the Pico is ready to accept button presses ❺, and you also toggle the onboard LED to OFF ❻. Then you start a `while` loop to monitor for button presses. Within this loop, you use a `for` loop to check whether any of the five buttons have a value of `0`, indicating the button is pressed. If so, you look up the note corresponding to that button in the `btnNotes` dictionary and play it using `play_note()` ❼. Once the note is done playing, you break out of the `for` loop and wait for 0.2 seconds ❽ before continuing with the outer `while` loop.

Running the Pico Code

Now you're ready to test your project! To run the code on the Pico, it's useful to install two pieces of software. The first is Thonny, an open source, easy-to-use Python integrated development environment (IDE), which you can download from ***https://thonny.org***. Thonny makes it easy to copy your project code to the Raspberry Pi Pico and manage files on the Pico. A typical development cycle is as follows:

1. Connect your Pico to your computer via USB.
2. Open Thonny. Click the Python version number in the bottom-right of the window and change the interpreter to **MicroPython (Raspberry Pi Pico)**.
3. Copy your code to Thonny and click the red **Stop/Restart** button to stop the code from running on the Pico. This will show the Python interpreter at the bottom of the IDE.
4. Edit your code in Thonny.
5. When you're ready to save the file, select **File▸Save As**, and you'll be prompted to save it on the Raspberry Pi Pico. The next dialog will also list the files on the Pico. Save your code as *main.py*. You can also use this dialog to right-click and delete existing files on the Pico.
6. After saving the file, press the extra push button you hooked up to the RUN pin on the Pico, and your code will start running.
7. Anytime you want to edit the code, click the **Stop/Restart** button in the IDE and Thonny will drop you to the Python interpreter on the Pico.

The other useful piece of software for working with the Pico is CoolTerm, which you can download from ***http://freeware.the-meiers.org***. CoolTerm lets you monitor the Pico's serial output. All print statements from your program will end up here. To use CoolTerm, ensure that you aren't "stopped" in Thonny. The Pico code should be in the running state, since the Pico can't connect to both Thonny and CoolTerm at the same time.

Once you have the code running, press the push buttons one by one and you'll hear the notes of a nice pentatonic scale coming out of your speaker. **Figure 12-8** shows the serial output for a typical session.

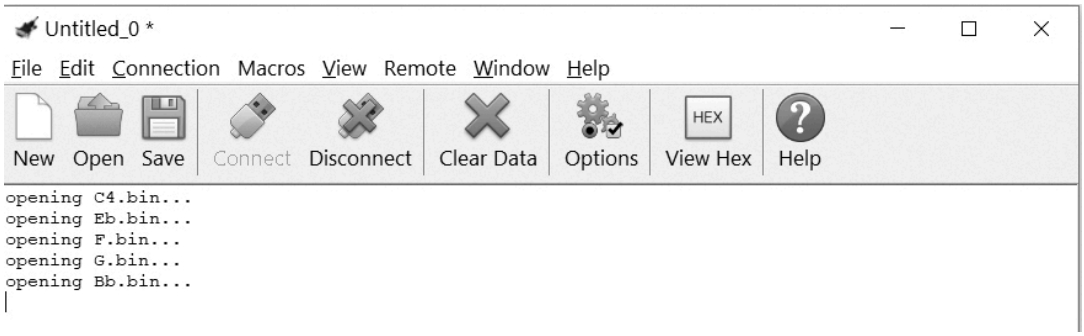

Figure 12-8: *A sample Raspberry Pi Pico output in CoolTerm*

See what melodies you can compose and play using your digital instrument's five push buttons!

Summary

In this chapter, you adapted your Karplus-Strong algorithm implementation from **Chapter 4** to run on a tiny microcontroller and built a digital musical instrument using a Raspberry Pi Pico. You learned how to run Python (in the form of MicroPython) on the Pico, as well as how to transmit audio data using the I2S protocol. You also learned about the limitations of adapting code from a personal computer to a resource-constrained device like the Pico.

Experiments!

1. The MAX98357A I2S board lets you increase the volume (gain) of the output. Look at the datasheet of this board and try to boost the sound coming from the speaker.
2. 2. The current implementation of `generate_note()` isn't very fast. It didn't matter that much for this project, because you generate

the notes only once. Still, can you make the method faster? Here are a few strategies to try:

1. Instead of using `append()` and `pop()` operations on the `buf` list, turn the list into a circular buffer by keeping tracking of the current position in the list and incrementing it using the modulo operation `%N`.
2. Use integer operations instead of floating point. You'll have to think about how the initial random values will be generated and scaled.

The Language Reference page in the MicroPython documentation (***https://docs.micropython.org***) has an article on maximizing the speed of your code. The documentation also suggests how you can test your results. First, define a function to measure timing:

```
def timed_function(f, *args, **kwargs):
    myname = str(f).split(' ')[1]
    def new_func(*args, **kwargs):
        t = time.monotonic()
        result = f(*args, **kwargs)
        delta = time.monotonic() - t
        print('Function {} Time = {:f} s'.format(myname, delta))
        return result
    return new_func
```

Then use `timed_function()` as a *decorator* for the function you want to time:

```
# generate note of given frequency
@timed_function
def generateNote(freq):
    nSamples = SR
    N = int(SR/freq)
    -- snip --
```

When you call `generateNote()` in your main code, you'll see something like this in the serial output:

```
Function generateNote Time = 1019.711ms
```

3. When you press a hardware push button, it doesn't just go from ON to OFF, or the reverse. The spring-loaded contacts inside the button bounce between ON and OFF multiple times in a fraction of a second, triggering multiple software events for one physical press of the button. Think about how this could affect your project, and then read up on *debouncing*, a class of techniques for mitigating the problem. What steps can you take to debounce your buttons?
4. When you press a button, a new note isn't played until the current note is done playing. How can you abruptly stop playing the current note when a new button is pressed and switch to playing the new note immediately?

The Complete Code

Here's the full code listing for this project.

```
"""

karplus_pico.py

Uses the Karplus-Strong algorithm to generate musical notes in a

pentatonic scale. Runs on a Raspberry Pi Pico. (MicroPython)

Author: Mahesh Venkitachalam

"""

import time

import array

import random

import os
```

```
from machine import I2S

from machine import Pin

# notes of a minor pentatonic scale

# piano C4-E(b)-F-G-B(b)-C5

pmNotes = {'C4': 262, 'Eb': 311, 'F': 349, 'G':391, 'Bb':466}

# button to note mapping

btnNotes = {0: ('C4', 262), 1: ('Eb', 311), 2: ('F', 349), 3: ('G', 391),

        4: ('Bb', 466)}

# sample rate

SR = 16000

def timed_function(f, *args, **kwargs):

    myname = str(f).split(' ')[1]

    def new_func(*args, **kwargs):

        t = time.ticks_us()

        result = f(*args, **kwargs)

        delta = time.ticks_diff(time.ticks_us(), t)

        print('Function {} Time = {:6.3f}ms'.format(myname, delta/1000))

        return result

    return new_func
```

```
# generate note of given frequency
# (Uncomment line below when you need to time the function.)
# @timed_function
def generate_note(freq):
    nSamples = SR
    N = int(SR/freq)
    # initialize ring buffer
    buf = [2*random.random() - 1 for i in range(N)]
    # init sample buffer
    samples = array.array('h', [0]*nSamples)
    for i in range(nSamples):
        samples[i] = int(buf[0] * (2 ** 15 - 1))
        avg = 0.4975*(buf[0] + buf[1])
        buf.append(avg)
        buf.pop(0)
    return samples
# generate note of given frequency - improved method
def generate_note2(freq):
    nSamples = SR
```

```
    sampleRate = SR
    N = int(sampleRate/freq)
    # initialize ring buffer
    buf = [2*random.random() - 1 for i in range(N)]
    # init sample buffer
    samples = array.array('h', [0]*nSamples)
    start = 0
    for i in range(nSamples):
        samples[i] = int(buf[start] * (2**15 - 1))
        avg = 0.4975*(buf[start] + buf[(start + 1) % N])
        buf[(start + N) % N] = avg
        start = (start + 1) % N
    return samples
def play_note(note, audio_out):
    "read note from file and send via I2S"
    fname = note[0] + ".bin"
    # open file
    try:
        print("opening {}...".format(fname))
```

```
    file_samples = open(fname, "rb")
except:
    print("Error opening file: {}!".format(fname))
    return
# allocate sample array
samples = bytearray(1000)
# memoryview used to reduce heap allocation
samples_mv = memoryview(samples)
# read samples and send to I2S
try:
    while True:
        num_read = file_samples.readinto(samples_mv)
        # end of file?
        if num_read == 0:
            break
        else:
            # send samples via I2S
            num_written = audio_out.write(samples_mv[:num_read])
except (Exception) as e:
```

```
        print("Exception: {}".format(e))

    # close file

    file_samples.close()

def create_notes():

    "create pentatonic notes and save to files in flash"

    files = os.listdir()

    for (k, v) in pmNotes.items():

        # set note filename

        file_name = k + ".bin"

        # check if file already exists

        if file_name in files:

            print("Found " + file_name + ". Skipping...")

            continue

        # generate note

        print("Generating note " + k + "...")

        samples = generate_note(v)

        # write to file

        print("Writing " + file_name + "...")

        file_samples = open(file_name, "wb")
```

```
        file_samples.write(samples)

        file_samples.close()

    def main():
        # set up LED
        led = Pin(25, Pin.OUT)
        # turn on LED
        led.toggle()
        # create notes and save in flash
        create_notes()
        # create I2S object
        audio_out = I2S(
            0,                  # I2S ID
            sck=Pin(0),         # SCK Pin
            ws=Pin(1),          # WS Pin
            sd=Pin(2),          # SD Pin
            mode=I2S.TX,        # I2S transmitter
            bits=16,            # 16 bits per sample
            format=I2S.MONO,    # Mono - single channel
            rate=SR,            # sample rate
```

```
    ibuf=2000,          # I2S buffer length
)
# set up btns
btns = [Pin(3, Pin.IN, Pin.PULL_UP),
        Pin(4, Pin.IN, Pin.PULL_UP),
        Pin(5, Pin.IN, Pin.PULL_UP),
        Pin(6, Pin.IN, Pin.PULL_UP),
        Pin(7, Pin.IN, Pin.PULL_UP)]
# "ready" note
play_note(('C4', 262), audio_out)
print("Piano ready!")
# turn off LED
led.toggle()
while True:
    for i in range(5):
        if btns[i].value() == 0:
            play_note(btnNotes[i], audio_out)
            break
    time.sleep(0.2)
```

```
# call main

if __name__ == '__main__':

    main()
```
